



Towards Automated Continuous Security Compliance

Florian Angermeir

fortiss

Germany

Blekinge Institute of Technology

Sweden

angermeir@fortiss.org

Fabiola Moyón

Siemens Technology

Germany

Technical University of Munich

Germany

fabiola.moyon@tum.de

Jannik Fischbach

Netlight Consulting GmbH

Germany

fortiss

Germany

jannik.fischbach@netlight.com

Daniel Mendez

Blekinge Institute of Technology

Sweden

fortiss

Germany

daniel.mendez@bth.se

Abstract

Context: Continuous Software Engineering is increasingly adopted in highly regulated domains, raising the need for continuous compliance. Adherence to especially security regulations – a major concern in highly regulated domains – renders *Continuous Security Compliance* of high relevance to industry and research.

Problem: One key barrier to adopting continuous software engineering in the industry is the resource-intensive and error-prone nature of traditional manual security compliance activities. Automation promises to be advantageous. However, continuous security compliance is under-researched, precluding an effective adoption.

Contribution: We have initiated a long-term research project with our industry partner to address these issues. In this manuscript, we make three contributions: (1) We provide a precise definition of the term continuous security compliance aligning with the state-of-art, (2) elaborate a preliminary overview of challenges in the field of automated continuous security compliance through a tertiary literature study, and (3) present a research roadmap to address those challenges via automated continuous security compliance.

CCS Concepts

• **Security and privacy** → **Software security engineering**; • **Software and its engineering** → **Software verification and validation**; **Process validation**.

Keywords

Continuous Security Compliance, Continuous Compliance, Continuous Software Engineering, Security Challenges, Security Compliance

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ESEM '24, October 24–25, 2024, Barcelona, Spain

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1047-6/24/10

<https://doi.org/10.1145/3674805.3690748>

ACM Reference Format:

Florian Angermeir, Jannik Fischbach, Fabiola Moyón, and Daniel Mendez. 2024. Towards Automated Continuous Security Compliance. In *Proceedings of the 18th ACM / IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM '24)*, October 24–25, 2024, Barcelona, Spain. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3674805.3690748>

1 Introduction

Over the last two decades, Continuous Software Engineering (CSE), entailing Agile and DevOps [9], transformed how software is developed, with the promise of creating software faster, providing value earlier, and, thus, resulting in higher project success rates. Some of the success factors are flexible adoption of requirements, early customer integration, enhanced team collaboration (e.g., between Dev, Ops, and Sec), and continuous experimentation and improvement [4, 8, 12]. In the light of highly regulated domains such as healthcare, critical manufacturing, or transport systems, the same benefits entail challenges for providers, seemingly rendering CSE and highly regulated environments incompatible. Providers in highly regulated domains have to adhere to regulations like laws (e.g., GDPR¹), standards (e.g., ISO 27001 [21]), internal policies, contracts or best practices to ensure the quality of their systems over disciplines including security, safety or privacy. Non-compliance with applicable regulations can negatively impact the quality of a system, with various risks for providers. These risks range from reputation damage over monetary loss² to competitive disadvantage [15]. Providers try to reduce those risks already in requirements engineering where regulations provide a source for requirements to be extracted, analyzed as vague and volatile subjects (e.g., [7, 26]), and implemented. To ensure compliance with regulations, companies must assess and demonstrate the effective fulfillment of identified requirements. In the context of CSE, requirements extraction, analysis, implementation, and demonstration (e.g., by verification) have to be performed frequently during development instead of following big-bang schedules, e.g. during releases [17], resulting in *Continuous Compliance*. However, manual compliance activities are too resource-intensive

¹<https://data.europa.eu/eli/reg/2016/679/oj>

²https://web.archive.org/web/20230708131758/https://edpb.europa.eu/news/news/2023/12-billion-euro-fine-facebook-result-edpb-binding-decision_en

to keep up with the ever-changing nature and high development velocity of CSE. Already in 2014, Fitzgerald and Stol acknowledged *Continuous Compliance* as a major challenge for highly regulated domains in adopting CSE [16]. Since then, more and more providers of software-intensive systems in highly regulated domains have adopted CSE, exacerbating the challenges and pressing need to overcome them.

Problem Statement. Automation promises to be advantageous. However, the multi-faceted field of *Continuous Compliance* is still vastly under-researched. Even though aspects such as automated compliance checking gained attention over recent years [2] with promising approaches such as [18, 19], the need for more extensive and structured research on automation approaches is highlighted by Castellanos Ardila et al. [10]: “It is difficult to guarantee industrial adoption when there is nonexistent or loosely coupled tool support [...]. Thus, it is crucial to provide adequate and complete tool support for automatically [*sic*] perform compliance checking” [10]. While automation approaches have gained attention over the last years, the compliance challenges of CSE in highly regulated domains and the structurization of such seem very much neglected. From a security compliance perspective, Moyón et al. [28] offer preliminary insights. However, further research seems necessary to provide an extensive landscape of challenges in employing CSE while adhering to security regulations (for now referred to as *Continuous Security Compliance*). Exacerbating the issue, to this day, little research has been performed on the requirements and their systematization for automation solutions in continuous compliance, even less *Continuous Security Compliance*. Instead, researchers and practitioners too often follow a solution-oriented approach [31] where they focus on solutions while leaving requirements underspecified and poorly understood. To our knowledge, Kellogg et al. [22], and Santilli et al. [38] are those that stand out in literature by describing five and 13 general requirements for *Continuous Security Compliance* (covering automation aspects) - in a structured way from an industrial perspective.

Objectives. With our research, we operate at the intersection between requirements engineering, security, and continuous software engineering. We aim to contribute to introducing automation in *Continuous Security Compliance* in a problem-driven manner. To this end, we formulate the following research goals (RG):

- RG1 understand the challenges of adhering to security compliance regulations in continuous software engineering projects
- RG2 understand requirements and constraints for automation in *Continuous Security Compliance* as suggested in literature and practice
- RG3 analyze the potential and limitations of automation as treatment towards the identified challenges and its implications for manual involvement
- RG4 develop and evaluate – along the previously determined potential and limitations – sensible solutions containing automation in industrial settings to treat respective challenges

Contributions. In this manuscript, we define the terms *Continuous Compliance* and *Continuous Security Compliance* as required for our research context. We provide the first insights into ongoing research through a tertiary study, in which we investigate and validate industrial challenges reported by practitioners. We conclude

by presenting our research roadmap towards achieving our objectives and bringing together requirements engineering, security, and continuous software engineering. One hope we associate with this contribution is to foster a critical reflection and discussion on contemporary challenges in automated *Continuous Security Compliance* and engage the research community to join our research endeavor.

Data Availability. The protocol of our literature study and the analyzed and processed data is shared in our online material [6].

2 State of Research

Based on the current state of research, we lay the foundation for future research by defining the term *Continuous Security Compliance* (CSC) in Section 2.1 and contributing to RG1 through an analysis of relevant challenges reported in the literature in Section 2.2.

2.1 Continuous Security Compliance (CSC)

The field of CSC has gained more interest over the last years. To effectively operate in this field, researchers need a definition they can identify with and refer to. While the phrase *Continuous Security Compliance* is used in some resources ([1, 5, 14, 36]), to our knowledge, it has only been defined explicitly once by Moyón et al. as “combining CC [*Continuous Compliance*] and CS [*Continuous Security*] through the holistic view of standardisation that spans across people, processes, and technology. Regulatory requirements are utilised to derive security activities and integrate security into a process while making it standards-compliant” [30]. This definition references two concepts, *continuous compliance* and *continuous security*, defined by Fitzgerald and Stol in the seminal framework for continuous software engineering *Continuous* * [16]. While the existing *Continuous Security Compliance* definition coined the field as part of the CSE research agenda, it lacks preciseness concerning terminology in the compliance field (e.g. reference to regulatory requirements and compliance to standards), and has limitations on the applicability for practitioners (e.g., security compliance solely mentioned as process security compliance, assuming product security compliance is a consequence of process compliance). An updated definition is required to improve understandability based on a broader analysis of the current state-of-art, beyond *Continuous* *, and, therefore, extend the concept’s applicability.

We aim for a *Continuous Security Compliance* definition that fits contemporary research while considering practice observations gathered during the last +5 years of our research on CSC, together with our industrial partners. To that end, we conducted an ad-hoc literature review (as defined by [35]) on the terminology related to *continuous compliance*. We reviewed 12 resources mentioning or defining continuous compliance, including theses and grey literature ([3, 11, 16, 20, 22, 25, 27, 34, 37–40]). We extracted relevant parts from each of the manuscripts, providing insights into the concepts covered in definitions and the actual definitions (see [6] for details). Most resources framed continuous compliance in light of their contribution. This confirms the observations by Santilli et al. [38], who independently analyzed 13 resources, of which three overlap with ours. Of the resources we reviewed, only two provided contribution-independent definitions. One is the original definition by Fitzgerald and Stol [16] from 2014, and the other was recently introduced by Santilli et al. [38]. Santilli et al. also scanned

the field for a proper definition and provided their version. They define continuous compliance as “the application of approaches and techniques to guarantee compliance with standards throughout a system’s lifecycle. Continuous compliance activities span the various development phases [...]”. Their definition covers the most relevant aspects we could distill from the other resources as well as from our own experience (1) Continuous execution of compliance activities (e.g. assessment, update of artifacts), (2) adherence to regulatory requirements, and (3) compliance over the entire development life-cycle. However, none of the newer definitions explicitly oriented on Continuous Software Engineering and its principles and goals, which is the fourth aspect we add to our definitions. In our experience, the goal of continuous compliance is as manifold as the goals of CSE ranging from the enablement of compliance in fast-paced development, over enablement of organizations to respond quickly to changing compliance requirements, to continuous improvement of the compliance state.

Given these four aspects, we define the term continuous compliance ourselves and base the definition of *Continuous Security Compliance* on it. The goal of the definitions is to not only provide a contribution-independent, yet concise updated definition but also to respect the background under which it was originally introduced: continuous software engineering. We propose to define *Continuous Compliance* as a set of practices to ensure compliance targets adherence to requirements derived from relevant regulatory sources, integrated holistically into the product development life-cycle, following continuous software engineering principles and goals.

The benefits of the updated continuous compliance definition are its alignment with current state-of-the-art practices, its shortness, agnosticism regarding the regulatory source and product development methodologies, as well as the explicit orientation on continuous software engineering principles and goals. The definition covers requirements from various regulatory sources such as standards, laws, policies, or contracts and is not restricted to a certain domain (e.g. security, safety) or compliance target (e.g. software development process). While the definition ensures that the original background (CSE) is preserved, it does not further stipulate the product development methodology. Additionally, given that large parts of regulated domains haven’t and might never fully transition to CSE, we do not prescribe the employed compliance process or its execution frequency. This flexibility enables practitioners to relate to *Continuous Security Compliance* already in the early stages of the continuous improvement journey in their organization (e.g. following the stairway to heaven model of [9]). The main downside of the flexibility in our definition is that it does not provide direct guidance toward implementing continuous compliance (e.g. compliance process execution frequency). However, this flaw also applies to existing definitions, and concrete guidance requires further in-depth research. With security as one application domain for continuous compliance, the before discussed advantages and disadvantages also apply to the CSC definition. For CSC the compliance target is usually products or processes and the requirements to comply with are derived from regulatory sources that are relevant to security and the respective organization.

Definition of Continuous Security Compliance

We define *Continuous Security Compliance* as a set of practices to ensure product and process adherence to requirements derived from relevant security regulatory sources, integrated holistically into the product development life-cycle, following continuous software engineering principles and goals.

2.2 Challenges in Continuous Security Compliance

Large parts of security compliance research focus on traditional software development. This state does not reflect the ever-growing adoption of Continuous Software Engineering (CSE) methodologies in the industry. Consequently, effective research is difficult without structured overviews of the challenges experienced in practice when following CSE principles while pertaining to security regulations. This state is exacerbated by the difficulties of cooperating with industry on such delicate matters and publishing the findings. One instance of such research is Moyón et al., who present 15 challenges experienced in three different companies [28].

Contributing to RG1, we research the pertinence of challenges on CSC in literature, validating and extending the challenges of Moyón et al. [28]. We performed a tertiary literature study on three major scientific databases: IEEE Xplore, ACM Digital Library, and Scopus. We followed Kitchenham [23] for the planning and execution of the literature study. The literature review protocol and other relevant artifacts for traceability are available in our online material [6]. The search initially yielded 110 hits, one of which was a duplicate. Manual filtering based on the abstract and the full text left three relevant manuscripts ([24, 29, 32]). Following Wohlin’s guidelines [44], we performed forward and backward snowballing following the references in each of the manuscripts and employing the citation feature of Google Scholar, yielding one more publication [33] (referenced by [29]).

The identified manuscripts investigated 15 security challenges in large-scale agile software development [32], 23 challenges while adopting DevSecOps [24], four security compliance challenges in agile software development [29], and 20 challenges while developing secure software using agile under the consideration of security compliance. In total, we extracted 62 challenges from those manuscripts.

Validation: Nine of the 15 challenges reported by Moyón et al. [28] (C1, C3, C4, C6, C7, C8, C9, C13, C14) could be found in literature. Leaving six challenges unvalidated. Those challenges are not necessarily irrelevant, as further research might validate them. The challenges of [28] and the number of challenges in the literature that validated them are listed in Table 1 (denoted as C1-C15). The pertinency of challenges C6, C7, C8, and C14 in literature (see Table 1) highlights and aligns with Moyón et al.’s perspective on the need for research about automatic continuous security compliance.

Extension: The remaining 47 challenges from the literature were used to extend the challenges given by Moyón et al. [28]. To this end, we performed a thematic analysis, analyzing and aggregating similar challenges across varying granularities into 12 common

new challenges (NC). The online material [6] details the analysis and aggregation steps, and Table 1 depicts all new challenges with the number of supporting challenges from literature (denoted as NC1-NC12). For illustration purposes, we briefly describe one such new challenge. *NC4: Security compliance evidence generation and documentation too resource consuming*. NC4 is based on three challenges that are thematically connected:

- “Security compliance requires documented evidence which increases delivery time” [29]
- “Security assessment favors detailed documentation” [33]
- “Teams need extra effort [...] to involve security experts and assessors to evaluate accuracy of documentation” [29]

As becomes apparent, security compliance evidence is the common subject here, with either the generation of evidence or the documentation of evidence being the activities that imply high resource demand (e.g., an increased delivery time or extra effort). Three challenges in literature could not be aggregated: “Customer readiness for applying frequent releases to production setup” [24], “Lacking guidance on the implementation of scaling agile frameworks” [32], and “Overcome security expert shortage at scale” [32].

Usage of Challenges & Preliminary Evaluation: Of the new challenges, the following seem to be relevant for automated CSC: NC1, NC2, NC4, NC5, NC7, and NC8. These challenges are part of the basis for our research roadmap detailed in Section 3.

As a preliminary evaluation, we interviewed a security expert with more than 10 years of security experience in the industry to evaluate the understandability of the challenges and get their perspective on the most relevant challenges for automated CSC. To that end, we took a snapshot of the existing relevant challenges at that time (C6, C7, C8, C14, NC1, NC2, NC4). We first asked for understandability of the new challenges and, afterward, which were the most relevant to them. All new challenges were perceived as easily understandable, and no clarification was necessary during the interview. The security expert perceived C6, NC1, and NC2 as the most pressing challenges in the subset providing indicators for the focus of future research (see Section 3).

i Summary

Nine challenges delineated by Moyón et al. [28] were validated in literature: C1, C3, C4, C6 - C9, C13, C14. The list of challenges was extended by 12 challenges. Challenges relevant for future automated CSC research: C6, C7, C8, C14, NC1, NC2, NC4, NC5, NC7, NC8.

3 Research Roadmap

This research has four goals, as outlined in Section 1. In the following, we will give an overview of the research environment and outline our research vision to address the research goals. Figure 2 summarizes the endeavor, highlighting research activities, the environments in which they are carried out, and research outcomes.

Table 1: Extended List of Validated Industry Challenges C = Industry Challenges by Moyón et al. [28]. NC = New Challenges from Literature Review

CHALLENGE		NO. OF RELATED CHALLENGES IN LITERATURE
Category 1: Security in Continuous Development		
C1	Perform threat modeling and consistently increment / adapt it throughout sprints	1
C2	Include security activities into continuous deployment	0
C3	Involve security aspects into continuous experimentation or prototyping	1
NC1	Lacking integration of security activities in development method	6
NC2	Changes to requirement, design or implementation break system security requirements	3
Category 2: Security in the Value Stream		
C4	Prioritization of security requirements vs. system functionalities	3
C5	Make security architecture visible in backlog and documentation	0
C6	Get security activities into the early feedback principle of DevOps	1
NC3	Security compliance is treated as conflicting objective	3
Category 3: Security Implementation Efficiency		
C7	Balance efforts for security: improving process compliance vs. improving product quality	2
C8	Involve security activities with minimal burden of lead time	2
NC4	Security compliance evidence generation and documentation too time consuming	3
NC5	Security compliance too resource-intensive	4
Category 4: Security Knowledge		
C9	Enable security knowledge and ownership in engineering teams	2
C10	Congregate experts to interchange examples about DevOps and security	0
NC6	Lacking security awareness of non-engineering stakeholders	2
Category 5: Security into CI/CD pipelines		
C11	Effective hardening of development, test and deployment environments	0
C12	Achieve efficient handling of security tool findings and involve into regular issues handling process	0
C13	Selection of security tools to run in the CI/CD pipeline	2
C14	Match security compliance requirements with working pipelines	1
C15	Ensure protection of CI/CD pipelines	0
NC7	Limitation of security tests	2
NC8	Complexity in provisioning of tools and environments	5
(New) Category 6: Security Strategy Success		
NC9	Insufficient leadership on security	5
NC10	Collaboration and team dependencies as break-points	5
NC11	Organization of roles and responsibilities	3
NC12	Transformation to DevSecOps	3

3.1 Research Environment

Our research is performed with an industry partner, a large enterprise operating in highly regulated domains, aligning and building

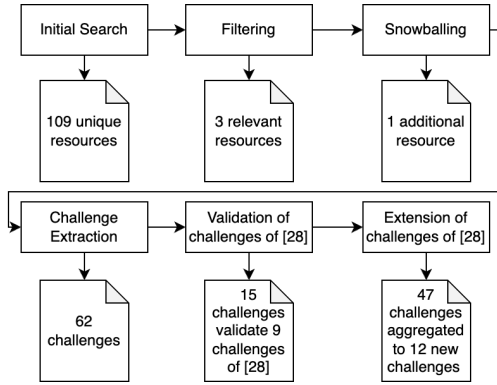


Figure 1: Literature Review Process

on top of their strong research foundation of the last 6 years (selected examples: [29, 30, 41, 42]) in the area of compliance to security standards applicable to the development and operation of software-intensive industrial automation and control systems. To ensure generalizability, we extend and validate our findings wherever possible with industrial companies partnering with Blekinge Institute of Technology as part of Software Engineering ReThought³, a large-scale academia-industry collaboration network entailing diverse industrial and academic backgrounds.

3.2 Research Vision

We structure the research according to the Design Science Methodology (DSM) [43], offering an iterative framework towards the problem investigation, treatment design, and treatment validation. DSM further enables the technology transfer to real-world application, closing the engineering cycle described by Wieringa. Since the DSM describes a cyclic approach, we will perform steps 2 and 3 iteratively, adapting to changing requirements, challenges, and technologies.

Step 0. Preliminary Challenges lays the foundation for the problem investigation of the DSM. We analyzed and reported the challenges in CSC (RG1) in Section 2.2. These preliminary challenges are used in the next step of our research.

Step 1. Challenges, Root Causes, Requirements contributes to the problem investigation and addresses parts of the treatment design of the DSM. We analyze the challenges in CSC and their root causes (RG1), the potential of these challenges for automation (RG3), and the requirements for automation in continuous security compliance (RG2).

In the first stage, we perform a survey to gather a broad perspective on the challenges and their automation potential from different stakeholders. This activity is performed internally at our industry partner and, to scale up, in parallel in the entire network of the large-scale academia-industry collaboration *Software Engineering ReThought*³ as well as with selected external partners. Based on the survey findings, in the second stage, we design and perform in-depth interviews with CSE stakeholders targeting the root causes of the challenges, their automation potential, and respective limitations and requirements for their automated treatments. In all of

³<https://rethought.se/>

these cases, we will thoroughly monitor literature (e.g. [13] reports on automation limitations, [38] lists relevant requirements).

Step 2. State of the Art Analysis and Solution Design to progress in the treatment design of the DSM. Together with our industry partner, we decide on a subset of challenges to focus on. To scan for available solutions to address those challenges, we will perform a technology screening through a systematic literature review. Given the existing gap between research and industry, we will additionally review the potentials, limitations, and use cases of available solutions on the market (such as Drata⁴, Vanta⁵, Scrut⁶) in the hope of better understanding and ultimately bridging this gap. We will continuously experiment to test the applicability of chosen automation approaches to the practical context. This shall provide a basis for our treatment design in which we will adopt existing approaches or design new ones (RG4). We then continue with the treatment validation in step 3.

Step 3. Treatment Validation to close the DSM cycle. We validate existing and newly developed treatments for (1) the satisfaction of the requirements identified before, (2) the impact on *Continuous Security Compliance*, and (3) the value added by the treatments to CSE projects. For that purpose, we not only integrate the treatments into the processes of CSE projects at our industry partner but also measure their impact on CSE principles and consequently on their development process. Further, we analyze with security experts (e.g. auditors) whether the treatments' outputs improve *Continuous Security Compliance*, e.g. provide valid evidence for compliance. This way, highly regulated organizations can integrate our proposed treatments into their projects while ensuring the treatment's acceptance by auditors.

i Summary

We perform this research in an industrial setting iteratively in three stages: (1) research of challenges in practice, (2) analysis and design of treatments for these challenges, and (3) validation of the treatment designs.

4 Threats to Validity

We describe the threats to validity structured by the three contributions reported in this manuscript. First, the definition of continuous security compliance is based on an ad-hoc literature review. To put it in Ralph's and Baltes' way: The limitations of ad hoc review are the "unsystematic nature" that might be subject to sampling bias and hinders replication [35]. However, given the large content-wise overlap in-between literature and also with our insights gained from the industrial environment, we are confident that the threat is a minor one.

Second, the extension and validation of challenges in continuous security compliance. As with any systematic (tertiary) literature review, the exclusion of relevant resources is a major threat. We experimented with and improved the search strategy in four iterations before we reviewed the title and abstract of each manuscript.

⁴<https://drata.com/>

⁵<https://www.vanta.com/>

⁶<https://www.scrut.io/>

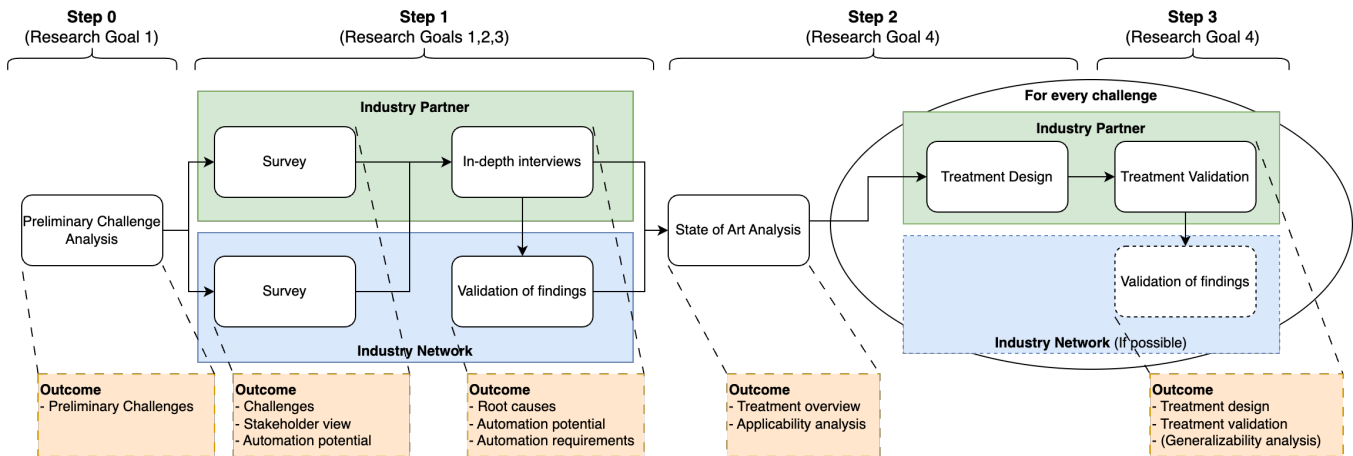


Figure 2: Summary of the Research Roadmap

We only dismissed manuscripts if we were certain about their irrelevance to us. Otherwise, we read the entire manuscript. After our analysis and synthesis, the first and the third authors discussed the challenges stemming from the extension to reduce experimenters' bias. In addition, we interviewed a security expert to ensure the validated and extended challenges were suitable for further research in practice.

Finally, the research roadmap describes our long-term research projects spanning multiple years which might be subject to change. As with any research roadmap, it is exposed to various (subjective) factors influencing its creation. Although our roadmap is rooted in industrially relevant challenges and corroborated with literature results, strengthening our confidence, we cannot guarantee its relevance. As an additional means to mitigate related threats, we intentionally share our roadmap with this paper to receive feedback and support from the research community.

5 Conclusion

The growing adoption of Continuous Software Engineering (CSE) in highly regulated domains raises the need for *Continuous Security Compliance*. Traditional compliance activities are slow, labor-intensive, and costly, rendering them unsuited for the fast-paced development and release of contemporary software development. Automation could contribute to transitioning security compliance from traditional, rigid, and slow to continuous, flexible, and fast-paced. However, to this day, automated *Continuous Security Compliance* is under-researched. This state forces companies operating in highly regulated domains to trade off between fast-paced development following CSE principles and reduced security risks by adhering to security regulations. We contributed to closing this gap by 1. providing a precise yet agnostic up-to-date definition of *Continuous Security Compliance* reflecting its four main components: continuous execution of compliance activities, adherence to relevant security regulatory sources, compliance over the entire development life-cycle, and its continuous software engineering background. 2. we offered a preliminary extension and validation of

industrial challenges at the intersection of requirements engineering, security compliance, and continuous software engineering. We also distilled six new challenges for future research of automated *Continuous Security Compliance*. 3. we outlined a research roadmap entailing the industrial research environment, research activities, and the outcomes required to treat the identified challenges successfully. With this manuscript, we hope to foster a critical reflection of the contemporary challenges and potential treatments of automated *Continuous Security Compliance* and engage the research community to join our research endeavor.

References

- [1] Javad Abed, Gurpreet Dhillon, and Sevgi Ozkan. 2016. Investigating Continuous Security Compliance Behavior: Insights from Information Systems Continuance Model. In *AMCIS '16*. 10 pages.
- [2] Abdel-Jaouad Aberkane, Geert Poels, and Seppe Vanden Broucke. 2021. Exploring Automated GDPR-Compliance in Requirements Engineering: A Systematic Mapping Study. *IEEE Access* 9 (5 2021), 66542–66559.
- [3] Muhammad Zaid Abrahams and Josef J Langerman. 2018. Compliance at Velocity within a DevOps Environment. In *ICDIM '18*. 94–101.
- [4] Abdullah Aldahmash, Andy M. Gravell, and Yvonne Howard. 2017. *Systems, Software and Services Process Improvement*. Chapter A Review on the Critical Success Factors of Agile Software Development, 504–512.
- [5] Arwa Alromaih, Yasser Ismail, and Wael Elmedany. 2022. Continuous compliance to ensure strong cybersecurity posture within digital transformation in smart cities. In *SCS '22*. 464–479.
- [6] Florian Angermeir, Jannik Fischbach, Fabiola Moyón, and Daniel Mendez. 2024. Towards Automated Continuous Security Compliance. <https://doi.org/10.6084/m9.figshare.25199225.v1>
- [7] Vanessa Ayala-Rivera and Liliana Pasquale. 2018. The Grace Period Has Ended: An Approach to Operationalize GDPR Requirements. In *RE'18*. 136–146.
- [8] Nasreen Azad and Sami Hyrynsalmi. 2023. DevOps critical success factors – A systematic literature review. *Information and Software Technology* 157 (5 2023), 14 pages.
- [9] Jan Bosch (Ed.). 2014. *Continuous Software Engineering*. Springer.
- [10] Julieth Patricia Castellanos Ardila, Barbara Gallina, and Faiz Ul Muram. 2022. Compliance checking of software processes: A systematic literature review. *Journal of Software: Evolution and Process* 34, 5 (5 2022), 36 pages.
- [11] Suresh Chari, Ian Molloy, Youngja Park, and Wilfried Teiken. 2013. Ensuring continuous compliance through reconciling policy with usage. In *SACMAT '13*. 49–60.
- [12] Tsun Chow and Dac-Buu Cao. 2008. A survey study of critical success factors in agile software projects. *Journal of Systems and Software* 81, 6 (6 2008), 961–971.
- [13] Fabiola Moyón Constante, Rafael Soares, Maria Pinto-Albuquerque, Daniel Mendez, and Kristian Beckers. 2020. Integration of Security Standards in DevOps Pipelines: An Industry Case Study. In *PROFES '20*. 434–452.

- [14] Sebastian Dännart, Fabiola Moyón Constante, and Kristian Beckers. 2019. An Assessment Model for Continuous Security Compliance in Large Scale Agile Environments. In *Advanced Information Systems Engineering*. 529–544.
- [15] João Faustino, Daniel Adriano, Ricardo Amaro, Rubén Pereira, and Miguel Mira da Silva. 2022. DevOps benefits: A systematic literature review. *Software: Practice and Experience* 52, 9 (9 2022), 1905–1926.
- [16] Brian Fitzgerald and Klaas-Jan Stol. 2014. Continuous Software Engineering and beyond: Trends and Challenges. In *RCoSE '14*. 1–9.
- [17] Brian Fitzgerald and Klaas-Jan Stol. 2017. Continuous software engineering: A roadmap and agenda. *Journal of Systems and Software* 123 (1 2017), 176–189.
- [18] Rajaa El Hamdani, Majd Mustapha, David Restrepo Amariles, Aurore Troussel, Sébastien Meeüs, and Katsiaryna Krasnashchok. 2021. A combined rule-based and machine learning approach for automated GDPR compliance checking. In *ICAIL '21*. 40–49.
- [19] Allenous Hayrapetian and Rajeev Raje. 2018. Empirically Analyzing and Evaluating Security Features in Software Requirements. In *ISEC '18*. 1–11.
- [20] IBM. 2013. Maintaining continuous compliance—a new best-practice approach. https://docs.media.bitpipe.com/io_11x/io_115656/item_894327/Maintaining%20continuous%20compliance.pdf
- [21] International Standards Organization. 2018. *Information technology - Security techniques - Information security management systems*. ISO Standard 27001.
- [22] Martin Kellogg, Martin Schäff, Serdar Tasiran, and Michael D. Ernst. 2020. Continuous Compliance. In *ASE '20*. 511–523.
- [23] Barbara Kitchenham. 2004. *Procedures for Performing Systematic Reviews*. Technical Report. Keele, UK and Eveleigh, Australia.
- [24] Rakesh Kumar and Rinkaj Goyal. 2020. Modeling continuous security: A conceptual model for automated DevSecOps using open-source software over cloud (ADOC). *Computer Security* 97, C (10 2020), 28 pages.
- [25] Ze Shi Li, Colin Werner, Neil Ernst, and Daniela Damian. 2020. GDPR Compliance in the Context of Continuous Integration. (2020). <https://doi.org/10.48550/arXiv.2002.06830> arXiv:arXiv:2002.06830v1
- [26] Aaron K. Massey, Richard L. Rutledge, Annie I. Anton, and Peter P. Swire. 2014. Identifying and classifying ambiguity for regulatory requirements. In *RE'14*. 83–92.
- [27] Marco Moscher. 2017. *Continuous Compliance Testing*. Master's thesis.
- [28] Fabiola Moyón, Florian Angermeir, and Daniel Mendez. 2024. Industrial Challenges in Secure Continuous Development. In *ICSE '24*. 3 pages.
- [29] Fabiola Moyón, Kristian Beckers, Sebastian Klepper, Philipp Lachberger, and Bernd Bruegge. 2018. Towards continuous security compliance in agile software development at scale. In *RCoSE '18*. 31–34.
- [30] Fabiola Moyón, Daniel Méndez, Kristian Beckers, and Sebastian Klepper. 2020. How to Integrate Security Compliance Requirements with Agile Software Engineering at Scale?. In *PROFES '20*. 69–87.
- [31] Daniel Méndez Fernández, Stefan Wagner, Klaus Lochmann, Andrea Baumann, and Holger de Carne. 2012. Field study on requirements engineering: Investigation of artefacts, project parameters, and execution strategies. *Information and Software Technology* 54, 2 (2 2012), 162–178.
- [32] Sebastian Nägele, Natalie Schenk, and Florian Matthes. 2023. The Current State of Security Governance and Compliance in Large-Scale Agile Development: A Systematic Literature Review and Interview Study. In *CBI '23*. 1–10.
- [33] Hela Oueslati, Mohammad Masudur Rahman, and Lotfi ben Othmane. 2015. Literature Review of the Challenges of Developing Secure Software Using the Agile Approach. In *ARES '15*. 540–547.
- [34] Simon Phipps and Stefano Zacchiroli. 2020. Continuous Open Source License Compliance. *Computer* 53, 12 (12 2020), 115–119.
- [35] Paul Ralph and Sebastian Baltes. 2022. Paving the way for mature secondary research: the seven types of literature review. In *ESEC/FSE'22*. 5 pages.
- [36] Rajesh Rompicharla and Bhaskar Reddy P. V. 2020. Continuous Compliance model for Hybrid Multi-Cloud through Self-Service Orchestrator. In *ICSTCEE '20*. 589–593.
- [37] Arstanaly Rysbekov. 2022. *Continuous Compliance: DevOps Approach to Compliance And Change Management*. Master's thesis.
- [38] Tiziano Santilli, Patrizio Pelliccione, Rebekka Wohlrab, and Ali Shahrokni. 2023. What is Continuous Compliance? *IEEE Software* (12 2023), 1–10.
- [39] Ali Shahrokni and Patrizio Pelliccione. 2022. Significance of Continuous Compliance in Automotive. In *EASE '22*. 272–273.
- [40] Andreas Steffens, Horst Lichter, and Marco Moscher. 2018. Towards Data-Driven Continuous Compliance Testing. In *SE '18*. 78–84.
- [41] Markus Voggenreiter, Florian Angermeir, Fabiola Moyón, Ulrich Schöpp, and Pierre Bonvin. 2022. Automated Security Findings Management: A Case Study in Industrial DevOps. In *ICSE-SEIP '22*. 11 pages.
- [42] Markus Voggenreiter and Ulrich Schöpp. 2022. Using a semantic knowledge base to improve the management of security reports in industrial DevOps projects. In *ICSE-SEIP '22*. 309–310.
- [43] Roel J. Wieringa. 2014. *Design Science Methodology for Information Systems and Software Engineering*.
- [44] Claes Wohlin. 2014. Guidelines for snowballing in systematic literature studies and a replication in software engineering. In *EASE '14*. Article 38, 10 pages.